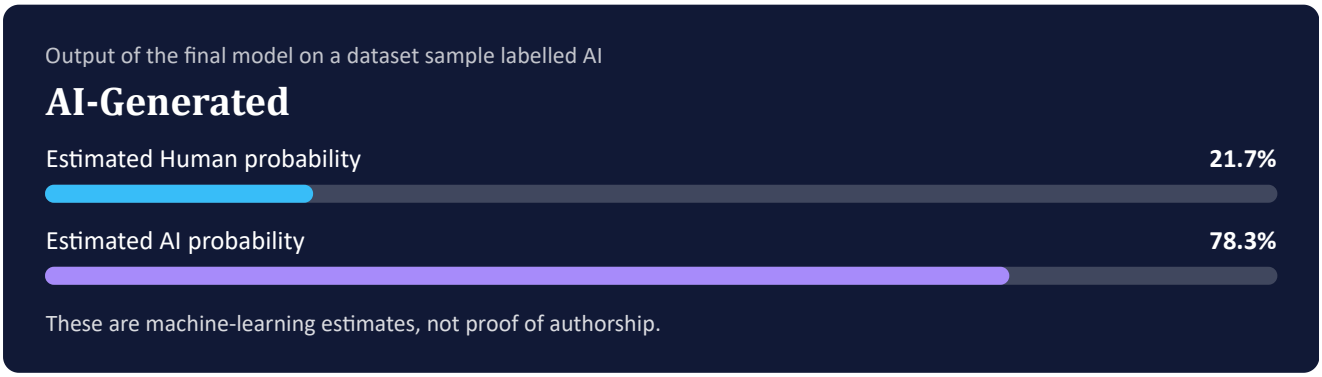
ECSCI24302 – Machine Learning Essentials
Project-Based Learning: Final Project Report

AI-Generated vs Human-Written Text Classification

An end-to-end Machine Learning project, from data exploration to model evaluation and deployment

Submitted by	[Name 1 – Enrollment No.] [Name 2 – Enrollment No.] [Name 3 – Enrollment No.]
Submitted to	[Instructor name]
Institution	[Institution and department]
Submission date	[Date]
Primary dataset	AI vs Human Text (Kaggle)



Contents

Abstract 3

1. Introduction 4

2. Problem Statement and Motivation 4

3. Dataset Description and Source 5

4. Data Understanding and EDA 5

5. Data Preprocessing 8

6. Feature Engineering and Feature Selection 8

7. Machine Learning Methodology 10

8. Baseline Model 11

9. Model Development 12

10. Hyperparameter Tuning 13

11. Results and Model Comparison 14

11.4 Benchmark: model quality 15

11.5 Benchmark: runtime performance 16

12. Error Analysis 19

13. Model Interpretation 19

14. Limitations 21

15. Conclusion and Future Work 21

References 22

Appendix A: Web Application 22

Appendix B: Reproducibility 23

Appendix C: Coverage of the Course Brief 24

Abstract

This project builds a Machine Learning system that estimates whether a passage of text was generated by AI or written by a human. It follows the complete workflow required by the course: problem definition, data acquisition and quality checks, exploratory analysis, preprocessing, feature engineering, baseline and model development, hyperparameter tuning, evaluation, error analysis, interpretation and demonstration.

The primary data is the Kaggle *AI vs Human Text* dataset, with 487,235 labelled texts, of which 487,231 remained after cleaning. Using TF-IDF features and a stratified 70:30 split, a most-frequent baseline reached 62.76% accuracy. Multinomial Naive Bayes reached 97.92%, tuned Logistic Regression 99.70% and tuned Linear SVM 99.95%.

Testing on paragraphs known to be human then showed a problem that these scores had hidden: the original Logistic Regression recipe flagged two of three such paragraphs as AI. Error analysis traced this to the removal of stop words, training only on long essays, a fixed 0.5 threshold and a human class containing only student essays. The final model addresses each cause. It combines word and character TF-IDF features, trains on essays, short snippets, pre-AI blogs and BBC news, and uses an AI threshold of 0.65 chosen on validation data. On held-out data, false AI flags on short human essay snippets fell from 8.5% to 0.4%, and on short blog and news snippets from 4.3% to 0.1%, while 96.8% of full-length AI essays were still detected.

Runtime benchmarks show the final model is practical on a laptop: a 400-word text takes a median of 4.6 ms to score directly and 7.0 ms through the REST API, and the service uses about 215 MB of memory. The model is served by a FastAPI backend and a React web application. Its outputs are estimates of class membership, not proof of authorship.

487,231
texts after cleaning

99.95%
best accuracy on the original test split (Linear SVM)

8.5% → 0.4%
false AI flags on short human essay snippets

7.0 ms
median API response for a 400-word text

Keywords: text classification, AI-generated text detection, TF-IDF, Logistic Regression, Linear SVM, decision threshold, error analysis, benchmarking.

1. Introduction

AI text generators can now produce fluent essays, summaries and articles in seconds. That creates a practical question for teachers, editors and anyone who reviews writing: how was this text produced? Machine Learning can look for statistical patterns that separate the two kinds of writing, but any answer it gives is an estimate. A system used carelessly can wrongly accuse a real writer.

This report documents an end-to-end project on this problem. The main lesson is that a high score on a test set is not enough on its own. Our first models looked nearly perfect, but only further testing and error analysis revealed how they behaved on human text outside the dataset's style. The final model was selected on that evidence rather than on the highest headline accuracy.

The report follows the structure set by the course brief. Sections 3 to 6 cover the data and features, Sections 7 to 10 the modelling, Sections 11 to 13 the results, benchmarks, errors and interpretation, and Sections 14 and 15 the limitations and conclusions. The appendices describe the web application and how to reproduce the work.

2. Problem Statement and Motivation

2.1 Problem statement

Given a passage of English text, predict whether it is AI-generated or human-written, and report an estimated probability for each class.

Table 1. Problem definition

Item	Definition
Input	Raw text as a string
Target	`generated`: 0 = human-written, 1 = AI-generated
Task	Supervised binary classification, because each training text carries a label and the output is one of two classes
Output	A predicted class, an estimated Human probability and an estimated AI probability
Expected outcome	A model that clearly beats a baseline, keeps false AI flags low, still detects most AI text, and can be demonstrated on new input

2.2 Motivation

The two possible errors do not cost the same. Missing an AI-written essay is undesirable, but wrongly flagging a person's own work can damage their reputation or grades. Published research has also found that AI detectors can be biased, with one study reporting that GPT detectors often misclassified non-native English writing as AI-generated [1]. For this reason we treat the false positive rate on human text as a primary concern, not only overall accuracy.

Text classification is also a good fit for the course. The problem is real, public labelled data exists, and it requires decisions at every stage of the pipeline.

3. Dataset Description and Source

3.1 Primary dataset

Table 2. Dataset summary

Property	Value
Name and source	AI vs Human Text, Kaggle, uploaded by shanegerami [2]
Link	kaggle.com/datasets/shanegerami/ai-vs-human-text
File	`AI_Human.csv`, 1.03 GB
Observations	487,235 before cleaning, 487,231 after cleaning
Columns	2: `text` (string, input) and `generated` (float, target)
Class counts before cleaning	305,797 human (62.76%) and 181,438 AI (37.24%)
Other variable types	No numeric or categorical inputs and no identifier column
Licence and usage	**[Confirm on the Kaggle dataset page]**

The dataset meets the brief's requirement for a meaningful, non-toy dataset. Its size also shaped the engineering: the full CSV was checked in chunks instead of being loaded at once, and the later experiments used subsamples to fit the memory available in a free notebook.

3.2 Additional human-written data

During model improvement we noticed that the dataset's human class consists of student essays. To give the model a wider view of human writing, two further Kaggle datasets were added. Both predate modern AI text generators, so their human labels are reliable.

Table 3. Additional datasets (human class only)

Dataset	Source	Period	Preparation
Blog Authorship Corpus	Kaggle: rtatman [3], based on [4]	2004	Removed the `urlLink` marker, normalised whitespace, kept 50–800 words with more than 60% letters, removed duplicates, sampled up to 20,000 posts with `random_state=42`
BBC Full Text Document Classification	Kaggle: alfathterry [5]	2004–2005	Normalised whitespace and removed duplicates

The combined extra text was shuffled and split into 65% training, 15% validation and 20% test. Row counts for these sets were printed by the notebook but not retained in its saved outputs, so they can be confirmed by re-running Step 25.

4. Data Understanding and EDA

4.1 Data quality

Quality checks ran over the complete dataset using chunked reading (50,000 rows per chunk) to control memory use.

Table 4. Missing-value, duplicate and validity analysis

Check	Method	Result
Missing text	`isna()` on all chunks	0
Missing labels	`isna()` on all chunks	0
Invalid labels	Values checked against {0, 1}	0
Exact duplicate texts	SHA-256 hash of each text across the full file	0
Repeated text-label pairs	Set of (text, label) pairs	0
Empty strings and non-string values	Direct comparison and type check	0
Whitespace-only texts	`str.strip() == ""`	4 (removed)
Same text with opposite labels, across train and test	Hash after lower-casing and normalising whitespace	2 pairs

The final check found near-identical pairs, a school lunch essay and a `Generic_City` essay, whose train and test copies carry opposite labels. Two pairs out of nearly half a million rows is negligible for performance, but it shows the labels are not perfect.

Reading samples of the raw text showed two more features of the data. Human essays contain anonymisation tokens such as `TEACHER_NAME` and `ADDRESS_NAME`. Texts in both classes also contain odd word substitutions, such as "hoe" for "how" in a human essay and "any" for "and" in an AI essay. We did not attempt to correct these, but tokens like these could become dataset-specific signals, which Section 13 confirms.

4.2 Class distribution (univariate)

EDA was carried out on a reproducible random sample of 20,000 records (`random_state=42`), which contained 12,440 human texts (62.2%) and 7,560 AI texts (37.8%), close to the full dataset's ratio.

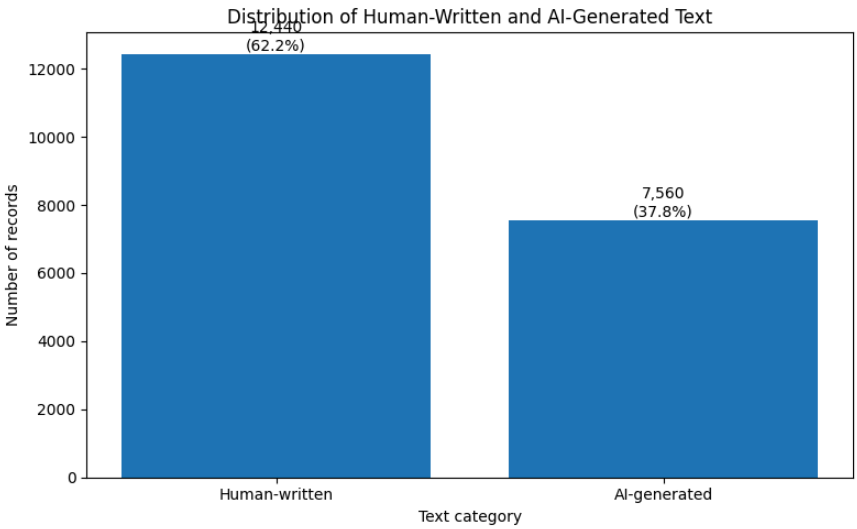


Figure 1. Class distribution in the 20,000-record sample (notebook output).

Observation. The classes are moderately imbalanced. A model that always predicts human would already score about 63% accuracy, so accuracy must not be the only metric, and the train-test split must be stratified.

4.3 Text length by class (bivariate)

Table 5. Text length statistics in the 20,000-record sample

Statistic	Human	AI	All
Records	12,440	7,560	20,000
Mean words	421.17	342.90	391.58
Median words	387	337	361
Minimum to maximum words	75 – 1,651	14 – 1,029	14 – 1,651
Mean characters	2,350.27	2,116.55	2,261.92

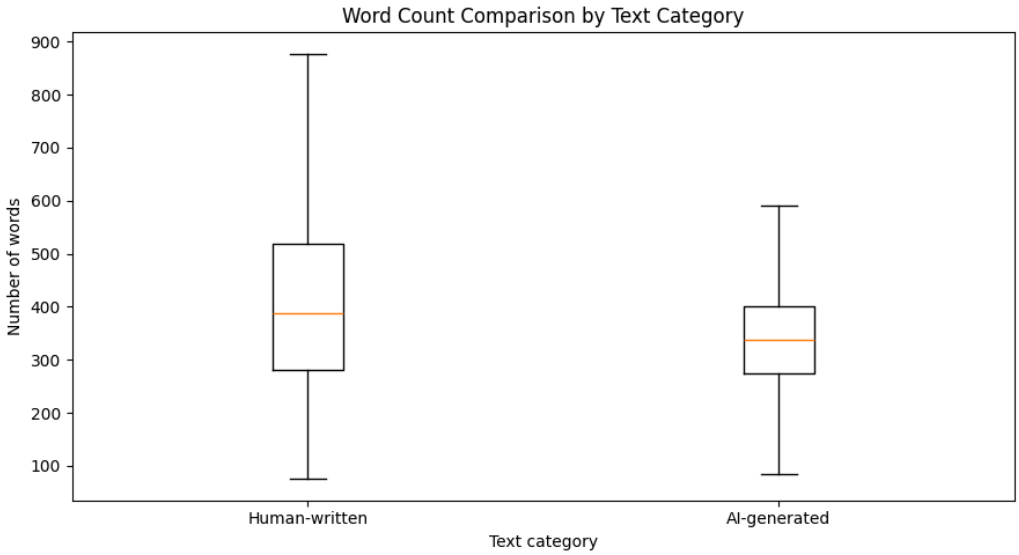


Figure 2. Word count by class (notebook output). The plot hides outliers.

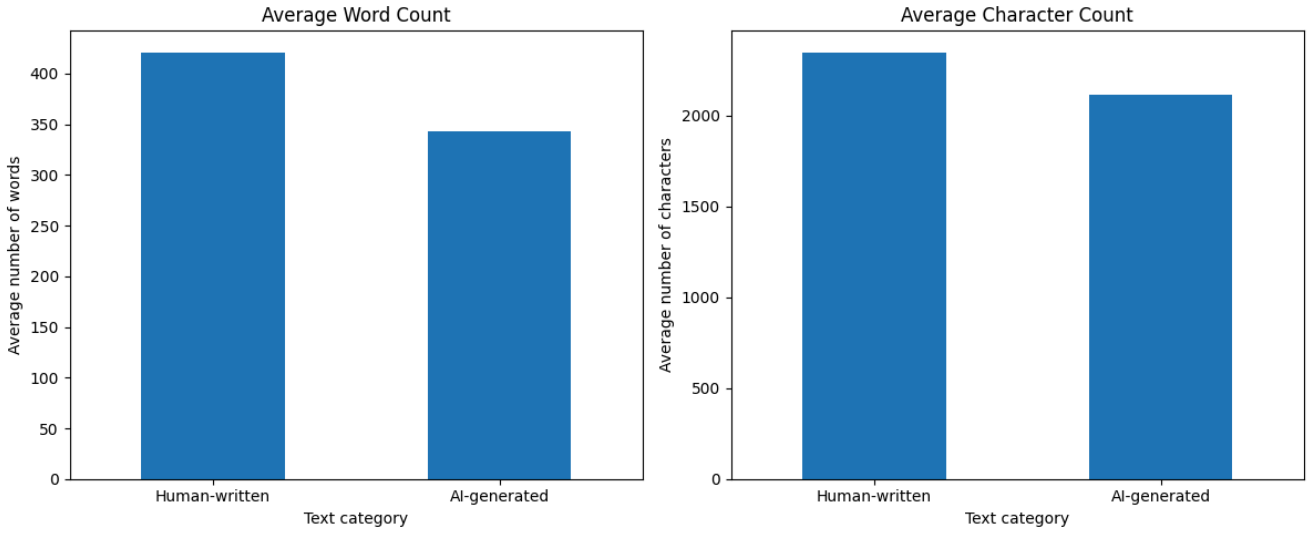


Figure 3. Average word and character counts by class (notebook output).

Observations.

- Human texts are longer on average, by about 78 words and 234 characters.
- Human text varies more in length, as the wider box shows.

- The two ranges overlap heavily, so length alone cannot identify the author. The model has to learn from the wording of the text itself.
- All full texts are essay length (median 361 words). A model trained only on these has little experience of short passages, which proved important later (Section 12).

4.4 Outliers

Word counts range from 14 to 1,651, well beyond the box plot's whiskers at both ends. The extreme values are genuine texts rather than data errors, and they were kept. TF-IDF vectors are normalised to unit length, which limits the influence of a long document, and length is not given to the model as a feature. Very short inputs are handled later through snippet training and a minimum length rule.

5. Data Preprocessing

Table 6. Preprocessing decisions and justification

Issue	Decision	Justification
Missing values	No imputation	None were found in text or labels
Whitespace-only rows	Removed 4 rows, leaving 487,231	They contain no information. The original file was preserved and a cleaned copy created
Duplicates	None removed from the main data	No exact duplicates exist. Duplicates were removed from the extra datasets
Outliers	Kept	They are genuine texts (Section 4.4)
Label type	Converted from float to `int8`	Labels are categories, and this saves memory
Categorical encoding	Not needed	The only input is text, and the target is already 0/1
Feature scaling	No separate scaler	TF-IDF rows are L2-normalised by default, which suits linear models
Text normalisation (final model)	`clean_text()`: lower-case, straighten curly quotes, collapse whitespace	Runs inside the pipeline, so training and prediction always treat text the same way

Unusual spelling and anonymisation tokens were left unchanged, so the model sees the text as supplied. Stemming and lemmatisation were not used because word forms carry style information.

6. Feature Engineering and Feature Selection

6.1 TF-IDF

Machine Learning models need numbers, so each text is converted to a vector with TF-IDF (Term Frequency – Inverse Document Frequency) [6]. A term scores highly when it appears often in a document but rarely across the collection:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t), \quad \text{with IDF}(t) = \ln((1 + n) \,/\, (1 + \text{df}(t))) + 1$$

Here *t* is a term, *d* a document, *n* the number of documents and *df(t)* the number of documents containing *t*. This is scikit-learn's default smoothed form [7]. With `sublinear_tf=True`, TF is replaced by 1 + ln(TF), so repeating a word gives diminishing returns. Each document vector is then L2-normalised.

6.2 Original feature recipe

Table 7. Original TF-IDF settings

Parameter	Value	Decision
`ngram_range`	(1, 2)	Single words and word pairs, to keep some phrase context
`stop_words`	english	Removed function words. This was reversed in the final model (Section 12)
`max_features`	50,000; 30,000 in tuning pipelines	Limits memory and time. The tuning pipelines used fewer features because each fit is repeated
`min_df`	2	Drops terms seen in only one document
`max_df`	0.95	Drops terms in nearly every document
`sublinear_tf`	True	Dampens the effect of repetition

The resulting training matrix has 341,061 rows and 50,000 columns. Only 0.34% of its cells are non-zero, so it is stored in sparse CSR format.

6.3 Final feature recipe

Error analysis (Section 12) showed the original recipe discarded style information. The final model joins two TF-IDF views with a `FeatureUnion`.

Table 8. Final model features (confirmed from the saved pipeline)

Parameter	Word view	Character view
Analyzer	Words and individual punctuation marks (`\b\w+\b|[^\w\s]`)	`char_wb` , characters within word boundaries
`ngram_range`	(1, 2)	(3, 5)
Stop words	Kept	Kept
`min_df` / `max_df`	3 / 0.9	5 / default
`max_features`	100,000	150,000
Preprocessor	`clean_text`	`clean_text`

Why these features. Function words such as "because", "very" and "the" carry much of a writer's style while saying little about the topic, so they should be kept. Punctuation tokens capture habits such as comma use. Character 3–5-grams capture spelling, word endings and punctuation patterns, and cope better with unseen word forms.

Snippet augmentation. Each training and validation essay also contributes one random passage of 15–150 words. This creates short examples with the same label, teaching the model to judge passages that are much shorter than a full essay.

6.4 Feature selection

Selection is built into the vectorizers rather than being a separate step. `min_df` removes rare terms, `max_df` removes near-universal ones, and `max_features` keeps the most frequent remaining terms. These rules use only document frequencies and are fitted on training data only. Beyond that, L2 regularisation in the classifier

shrinks the weights of less useful features. A statistical filter such as chi-squared was not used, because linear models with regularisation handle high-dimensional sparse features well.

7. Machine Learning Methodology

7.1 Workflow

The project had two phases. Phase A built and compared models on the full split. Phase B responded to what error analysis found.

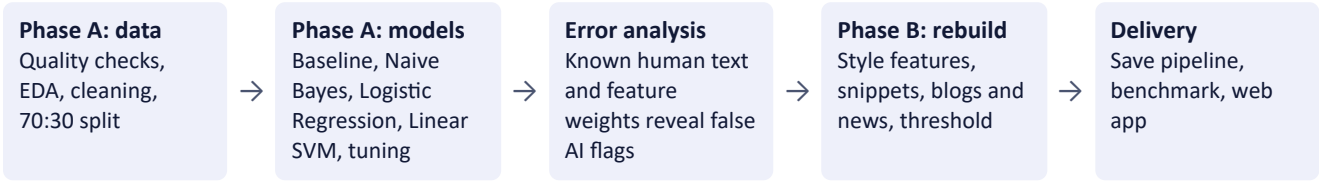


Figure 4. The project workflow.

7.2 Data splits

Table 9. How data was divided

Phase	Split	Records	Purpose
A: full data	Training (70%)	341,061	Model fitting and 3-fold cross-validation
	Test (30%)	146,170	Final evaluation of Phase A models (91,740 human, 54,430 AI)
B: subsample	Training	40,000 essays	From the Phase A training part; 80,000 rows with snippets, plus extra human text
	Validation	15,000 essays	Threshold selection, with snippets and extra human validation text
	Test	20,000 essays	From the Phase A test part, in full and snippet versions, plus extra human test text

All splits are stratified where labels are mixed and use `random_state=42`. Train and test indexes were checked and do not overlap.

7.3 Why these metrics

Table 10. Evaluation metrics

Metric	Definition	Why it is used here
Accuracy	(TP + TN) / all	Easy to read, but flattered by imbalance, so it is never used alone
Precision (AI)	TP / (TP + FP)	When the model says AI, how often is it right? This protects human writers
Recall (AI)	TP / (TP + FN)	How much AI text is caught
Macro F1	Mean of each class's F1	Counts both classes equally, despite the 63:37 split. Used as the tuning score
ROC-AUC	Area under the ROC curve	Measures ranking quality across all thresholds
False flag rate	FP / (FP + TN)	Share of human text wrongly labelled AI. The key safety metric in Phase B
Confusion matrix	Counts of each outcome	Shows the two error types separately

7.4 Preventing data leakage

- Vectorizers were fitted on training data only and merely applied to the test set.
- During tuning, TF-IDF sits inside a `Pipeline`, so each cross-validation fold learns its own vocabulary from its own training portion.
- The decision threshold was chosen on validation data, not test data.
- Phase B test records come from the Phase A test part, so they were never used for training.
- No feature contains target or future information, since the only input is the text.

One weakness remains: held-out data was used to compare versions of the model during Phase B. Section 14 discusses this.

8. Baseline Model

The baseline is a `DummyClassifier(strategy="most_frequent")`, which ignores the text and always predicts the majority class, human.

Table 11. Baseline results on the test set (146,170 records)

Class	Precision	Recall	F1	Support
Human-written	0.63	1.00	0.77	91,740
AI-generated	0.00	0.00	0.00	54,430
Macro average	0.31	0.50	**0.39**	146,170

Accuracy is 62.76%, yet the baseline catches no AI text at all, which is why macro F1 of 0.39 is a fairer reference than accuracy. Any useful model must clearly beat both figures.

9. Model Development

9.1 Models selected

Three algorithms were selected. All are linear models that handle large, sparse TF-IDF matrices efficiently, which matters at 341,061 rows and tens of thousands of features.

Table 12. Algorithms and why they suit the problem

Model	How it works	Why it was chosen
Multinomial Naive Bayes (`alpha=0.1`)	Applies Bayes' rule, assuming features are independent given the class	Very fast, a classic text baseline, and a check on whether more complex models add value
Logistic Regression (`liblinear`) [8]	Learns a weight per feature and applies the sigmoid function to their sum, minimising log loss	Gives probabilities through `predict_proba()` and has readable weights
Linear SVM (`LinearSVC`) [9]	Finds the separating hyperplane with the widest margin, minimising hinge loss	Typically strong on high-dimensional text, though it outputs distances rather than probabilities

$$P(\text{AI} \mid x) = \sigma(w \cdot x + b), \quad \sigma(z) = 1 / (1 + e^{-z})$$

9.2 Phase A training

Naive Bayes and an untuned Logistic Regression (`C=1.0`) were first trained on the 50,000-feature matrix. Linear SVM (`C=1.0` , `max_iter=2000`) was trained on the same features. Pipelines with 30,000 features were then tuned for Logistic Regression and Linear SVM (Section 10). Naive Bayes reached 97.92% accuracy and untuned Logistic Regression 99.71%, a large gain over the baseline.

9.3 Phase B: improved models

Phase B retrained on subsamples, for three reasons: the richer features need much more memory, this was run on a free notebook, and using the same subsample allowed a fair comparison between old and new recipes.

1. **Old recipe.** The Step 15 settings were retrained on the 40,000 essays to act as a reference.
2. **First improved model.** The word and character features of Section 6.3, with `C=0.5` , trained on the 40,000 essays and their snippets.
3. **Final model (v2).** The same recipe retrained with the extra blog and BBC text added to the human class, along with their snippets.

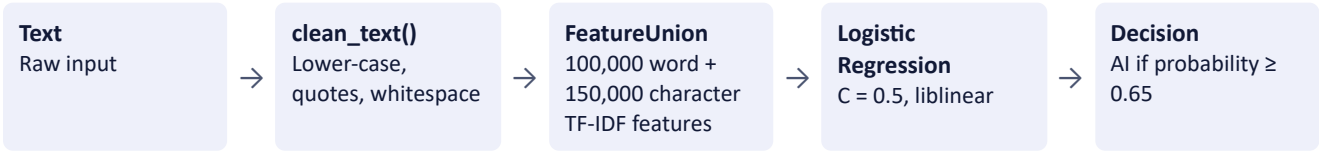


Figure 5. The final model's prediction pipeline.

The saved final model is `improved_lr_pipeline.joblib` , with its threshold and minimum length in `improved_model_config.json` . It was saved with scikit-learn 1.6.1, which the application pins for compatibility. We confirmed it is the final version by reproducing the notebook's reported dry-test scores from the saved file.

10. Hyperparameter Tuning

Logistic Regression and Linear SVM were tuned with `GridSearchCV`, using stratified 3-fold cross-validation with shuffling on the training set only, macro F1 as the score, and refitting the best model on all training data. The hyperparameter is `C`, the inverse of regularisation strength: a smaller C applies stronger regularisation and produces simpler weights, which reduces overfitting. One Logistic Regression fit took about 3.6 minutes, so the grid was deliberately small.

Table 13. Search space, selected parameters and cross-validation results

Model	Search space	Selected	Mean CV macro F1
Logistic Regression	C ∈ {0.5, 1.0}	C = 1.0 (from the saved tuned pipeline)	[re-run notebook cell 28]
Linear SVM	C ∈ {0.5, 1.0}	[re-run notebook cell 29]	[re-run notebook cell 29]
Multinomial Naive Bayes	Not tuned	alpha = 0.1	Not applicable
Final model (v2)	Not grid-searched	C = 0.5, more regularisation for 250,000 features	Threshold chosen on validation instead

10.1 Decision threshold

The decision threshold is also a tunable setting, and in this problem it has a direct effect on who gets flagged. A first attempt set it at the 99th percentile of human validation scores. This was replaced with a rule that keeps detection explicitly in view: from 0.50 to 0.95 in steps of 0.05, take the highest threshold that still catches at least 90% of AI text on validation data. For the final model, this gives **0.65**.



Figure 6. The notebook's three decision zones for the estimated AI probability.

In the notebook, a score from 0.50 up to the threshold is reported as Uncertain, and texts under 50 words are never labelled AI. The web application currently simplifies this to two labels: scores below 0.65 display as Human-Written, and texts under 50 words are rejected with a clear message.

11. Results and Model Comparison

11.1 Model comparison

Table 14. Final model comparison on the 146,170-record test set

Model	Best hyperparameters	Validation	Accuracy	Macro precision	Macro recall	Macro F1	ROC-AUC	Observations
Dummy baseline	most_frequent	Not applicable	0.6276	0.31	0.50	0.39	n/a	Never predicts AI
Naive Bayes	alpha = 0.1	Not tuned	0.9792	0.9793	0.9762	0.9777	0.9976	Fastest; lowest of the three
Tuned Logistic Regression	C = 1.0	[CV F1]	0.9970	0.9974	0.9962	0.9968	0.9999	Gives real probabilities
Tuned Linear SVM	[C]	[CV F1]	**0.9995**	**0.9995**	**0.9993**	**0.9994**	**1.0000**	Highest scores; no probabilities

The final v2 model is not in this table because it was evaluated on the Phase B test sets with a different question in mind: false flags and detection at its threshold. Section 11.3 reports it. Tuned Logistic Regression scored 0.9970 against 0.9971 for the untuned model. The tuned pipeline used fewer features (30,000 against 50,000), so tuning did not produce a meaningful gain here.

11.2 Confusion matrices

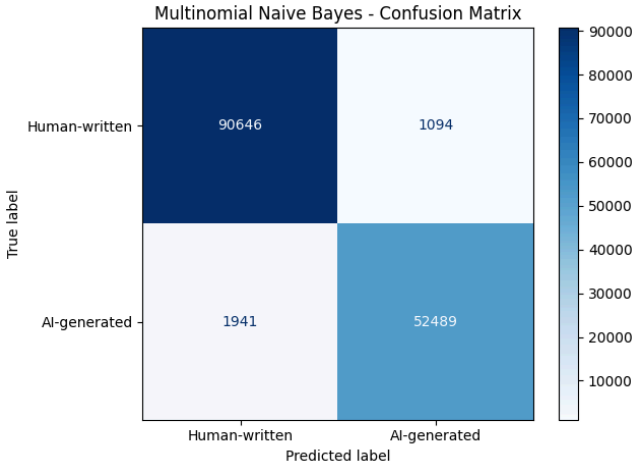


Figure 7. Multinomial Naive Bayes (notebook output).

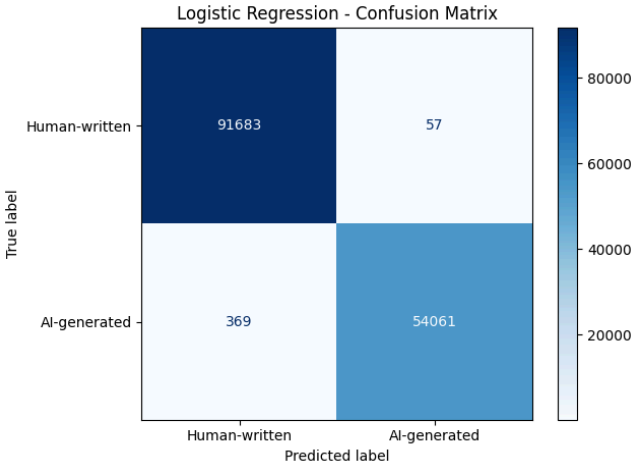


Figure 8. Logistic Regression, C = 1.0, 50,000 features (notebook output).

Table 15. Error rates derived from the confusion matrices

Model	Human flagged as AI	False flag rate	AI missed	Miss rate	AI precision
Multinomial Naive Bayes	1,094	1.19%	1,941	3.57%	97.96%
Logistic Regression (C = 1.0)	57	0.06%	369	0.68%	99.89%

The tuned Linear SVM's confusion matrix is not in the notebook's saved outputs. **[Re-run notebook cell 31 and add it here]**

11.3 Improvement results (Phase B)

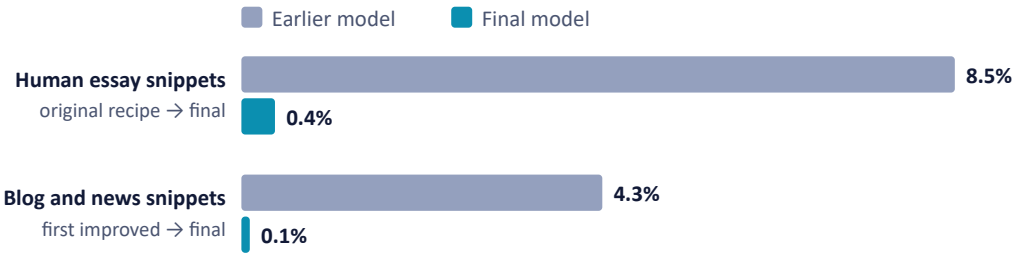


Figure 9. Short human text wrongly flagged as AI on held-out data (notebook results). Each pair compares a different earlier model, as labelled.

Table 16. Dry test on new paragraphs (AI probability, %)

Text	Original recipe	Final model	Change
AI samples 1–3	99.0 – 99.7	99.4 – 100.0	Still labelled AI
Human sample 1	27.9	2.2	Still labelled human
Human sample 2	68.2	14.8	Fixed: was wrongly labelled AI
Human sample 3	65.1	27.3	Fixed: was wrongly labelled AI

The final model still detects 96.8% of full-length AI essays at its threshold. This is lower than the near-perfect figures in Section 11.1 and is the price of a far lower false flag rate on human text, though the two figures come from different models, data sizes and thresholds, so they are not directly comparable.

11.4 Benchmark: model quality

This benchmark compares every model against the baseline on the shared test set, then shows the final model against the earlier recipes on the safety question.



Figure 10. Test accuracy on 146,170 records, with the axis starting at 0%.

Table 17. Gain over the baseline on the 146,170-record test set

Model	Accuracy	Accuracy gain over baseline	Macro F1	Macro F1 gain over baseline
Dummy baseline	62.76%	–	0.39	–
Naive Bayes	97.92%	+35.16 points	0.9777	+0.59
Tuned Logistic Regression	99.70%	+36.94 points	0.9968	+0.61
Tuned Linear SVM	99.95%	+37.19 points	0.9994	+0.61

We also ran every labelled example available without the full dataset through the saved final model. These are the three AI and three human dry-test paragraphs, the five human examples in the notebook's Step 23, and the two dataset samples used by the web app.

Table 18. Benchmark of the saved final model on 13 labelled examples

Example	Expected	Words	AI probability	Web app result
Dry test AI 1	AI	96	100.0%	AI-Generated
Dry test AI 2	AI	85	99.9%	AI-Generated
Dry test AI 3	AI	84	99.4%	AI-Generated
Dry test human 1	Human	99	2.2%	Human-Written
Dry test human 2	Human	103	14.8%	Human-Written
Dry test human 3	Human	59	27.3%	Human-Written
Notebook human example 1	Human	17	8.9%	Rejected (under 50 words)
Notebook human example 2	Human	28	3.3%	Rejected (under 50 words)
Notebook human example 3	Human	35	3.1%	Rejected (under 50 words)
Notebook human example 4	Human	43	1.1%	Rejected (under 50 words)
Notebook human example 5	Human	43	2.7%	Rejected (under 50 words)
Dataset sample (labelled human)	Human	157	0.2%	Human-Written
Dataset sample (labelled ai)	AI	120	78.3%	AI-Generated

Reading this benchmark. All 13 fall on the correct side, with every AI example at or above the 0.65 threshold and every human example below 0.50. This is a sanity check, not an accuracy measurement. The set is small, and some human paragraphs were known problem cases that prompted the improvements, so they are not independent of the final model. The held-out figures in Section 11.3 are the stronger evidence.

11.5 Benchmark: runtime performance

We measured the saved final model and the live FastAPI service directly on one machine. The earlier models are no longer in the project folder, so they could not be included.

Table 19. Benchmark environment

Item	Value
Hardware	13th Gen Intel Core i7-1360P, 16 logical processors, 15.6 GB RAM, laptop
Software	Windows 11, Python 3.12.7, scikit-learn 1.6.1, NumPy 2.5.4, joblib 1.6.0
Test text	Real sentences from the dry-test paragraphs and dataset samples, randomly joined to each target length with a fixed seed
API setup	One Uvicorn worker on localhost, without auto-reload; a warm-up of 20 requests before timing

9.6 MB
model file on disk

2.0 s
median model load time (5 runs)

215 MB
API server memory, stable after 500 requests

3.1 s
from starting the server to ready

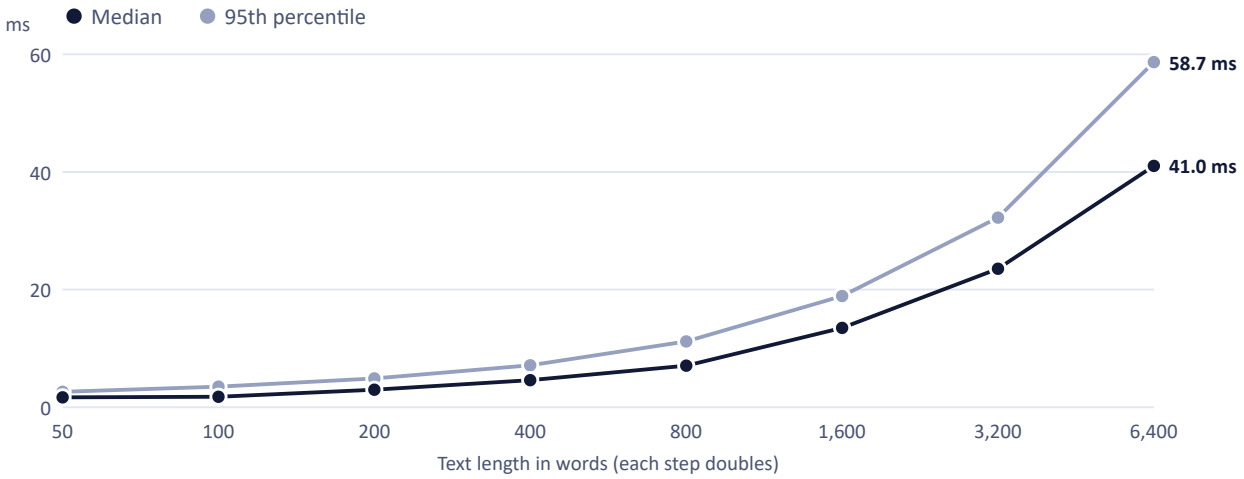


Figure 11. Direct prediction time for one text, measured on the saved final model.

Table 20. Direct prediction time for one text, single process

Words	Characters	Repeats	Median (ms)	95th percentile (ms)
50	319	500	1.7	2.6
100	550	500	1.8	3.5
200	1,199	400	3.0	4.9
400	2,373	300	4.6	7.1
800	4,667	200	7.1	11.2
1,600	9,577	150	13.5	18.9
3,200	19,354	100	23.6	32.2
6,400	37,545	60	41.0	58.7

Table 21. Batch throughput, with 400-word texts

Batch size	Median time (s)	Texts per second
1	0.004	231
10	0.038	266
100	0.311	321
1,000	3.196	313

Table 22. REST API response time, one client, 300 requests each

Endpoint and input	Median (ms)	95th percentile (ms)
GET /health	1.0	1.4
POST /predict, 100 words	3.9	5.9
POST /predict, 400 words	7.0	10.1
POST /predict, 1,600 words	12.6	19.3

Table 23. API under concurrent clients: 400-word texts, one server worker, about 800 requests

Clients	Requests	Requests per second	Median (ms)	95th percentile (ms)	Errors
1	800	107	9.0	14.4	0
4	800	117	31.1	45.6	0
8	800	123	58.2	81.2	0
16	800	95	137.6	187.3	0

Reading these results.

- Prediction time grows roughly in line with text length. At the API's 50,000-character limit it is still well under a tenth of a second by extrapolation from the 37,545-character measurement of 41.0 ms.
- Most of the startup cost is a single model load of about 2.0 seconds. After that, memory stays stable: 215 MB when idle and 215 MB after 500 further requests.
- With one worker, throughput reaches about 123 requests per second, while response times rise as clients queue. No request failed. The load generator ran in Python on the same machine, so these throughput figures are conservative, and they measure this laptop rather than a server.
- This is more than sufficient for a single-user demonstration, and the single-machine figures should be treated as indicative rather than as a guarantee.

11.6 Final model selection

Table 24. Why Logistic Regression v2 was selected

Criterion	Tuned Linear SVM	Logistic Regression v2 (final)
Best evidence	99.95% accuracy on the essay test split	0.4% false flags on short essays; 96.8% of AI essays caught
Probability estimates	No `predict_proba()`, margin scores only	Yes, as the application requires
Training text	Full essays only	Essays, short snippets, blogs and news
Decision rule	Fixed sign of the margin	Threshold of 0.65 chosen on validation
Interpretability	Linear weights	Linear weights over word and character features
Computation	Linear, 30,000 features	Linear, 250,000 features, 9.6 MB, 4.6 ms for 400 words

The SVM has the higher score on its test split, but these figures come from different test conditions and are not head-to-head. The decision rests on suitability: the application needs probability estimates, and the evidence shows that the final model greatly reduces false flags on human text. Its runtime cost is low enough for interactive use.

12. Error Analysis

12.1 What the test set could not show

After Phase A, the original recipe was tried on paragraphs known to be human. Two of three were labelled AI, with probabilities of 68.2% and 65.1%. A formal test sentence using "additionally" and "essential" received a displayed AI probability of 100.00%, while an informal sentence received 18.86%. Together these suggested that the model was reacting to formality and vocabulary rather than authorship.

12.2 Why the model made these errors

Table 25. Causes of false AI flags and the fix for each

#	Cause	Why it caused errors	Fix applied
1	Stop-word removal	Function words carry most of the style signal, so the model leaned on topic and formal words	Keep function words and punctuation, and add character n-grams
2	Long essays only	Training texts have a median of about 361 words, so short inputs gave overconfident guesses	Add random 15–150-word snippets to training
3	Fixed 0.5 threshold	Slightly formal text tipped straight into AI	Choose 0.65 on validation and define an Uncertain zone
4	Student essays only	Human text of any other style was unfamiliar	Add pre-AI blogs and BBC news

12.3 Other findings

- **Dataset patterns.** The strongest Human features in the Linear SVM include *venus* and *student_name*. The first looks like an essay topic and the second an anonymisation token. Neither says anything general about human writing, which shows the model also learned details of this dataset.
- **Label conflicts.** The two train-test pairs of identical text with opposite labels show a small, unavoidable level of label noise.
- **Linear SVM misclassifications.** The notebook's Steps 18 and 19 inspect the SVM's incorrect test predictions and their text patterns. **[Re-run notebook cells 31–33 and add the error counts and examples]**

12.4 Result of the fixes

Measured on held-out data, false flags fell sharply (Section 11.3). Accuracy on same-source essays alone could not have shown this problem, which is why testing outside the dataset's style became a required step.

13. Model Interpretation

Both Logistic Regression and Linear SVM are linear, so each feature's weight shows the direction and strength of its effect. A positive weight pushes towards AI and a negative weight towards human.

13.1 Original Linear SVM

Table 26. Strongest features in the tuned Linear SVM

Direction	Features
Towards AI	additionally, essay, important, essential, firstly
Towards Human	people, venus, student_name, probably, percent

The AI-associated words are common in formal writing, which helps explain why well-written human text was wrongly flagged.

13.2 Final model

The weights below were read directly from the saved final model, which has 250,000 features and an intercept of −1.03. The negative intercept means a neutral text starts with a lean towards human.

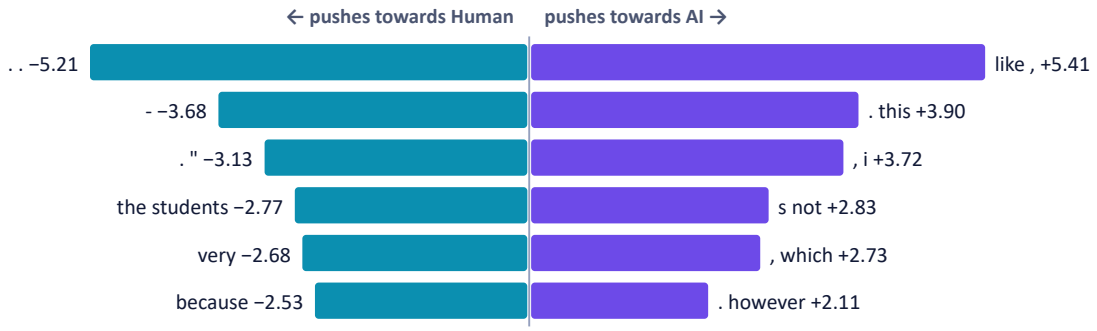


Figure 12. Strongest word features in the saved final model and their weights. Tokens with spaces are word pairs, including punctuation.

Table 27. Strongest character features in the final model

Direction	Character n-grams (weight)
Towards AI	`like,` (3.43), `ike,` (3.17), `y,` (1.96), `e,` (1.88), `s,` (1.86)
Towards Human	`...` (−1.91), ` so` (−1.73), ` go` (−1.65), ` very` (−1.60), `becau` (−1.53)

Meaningful patterns.

- Punctuation and function words now dominate, as intended. Formal topic words no longer lead the list.
- Comma use is prominent on the AI side, for example "like ," and ", which". Repeated periods, hyphens, quotation marks and brackets lean human.
- "like ," is the strongest AI feature. Some AI samples in the data are conversational (the web app's AI sample includes "Like, let's say"), which may explain this, but it could mislead on casual human writing.
- "the students" still leans human, showing the essay topics have not disappeared completely.

These weights are correlations in this particular training data. They describe the dataset, not universal rules about AI writing, and a single feature never decides a result.

14. Limitations

Table 28. Limitations and their effect

Limitation	Effect
AI examples are essay-only	Blogs and news were added as human only, so the model could learn that a non-essay style means human. AI-written blogs or news could be missed. This is the most important limitation
Held-out data reused	Phase B used the same held-out data to compare versions, which the brief warns against. The improvement figures may be slightly optimistic. A fresh test set would be needed for a final, unbiased measure
Same-source test split	Phase A scores near 100% say little about other kinds of writing
Generators change	Newer AI models may write differently from those behind the training data
Subsampled training	The final model used 40,000 essays rather than all 341,061 training records, to fit memory limits
Small dry test	Six paragraphs (thirteen in the benchmark) show direction only
Limited tuning	Only two values of C were searched, and the final model's C was set by design
Short and edited text	Very short text is unreliable. Human-edited AI text, or AI-edited human text, does not fit either class
English only	The training data is in English
Probability meaning	Scores show model confidence and have not been checked for calibration. They are not the percentage of a text written by AI
Application simplification	The web app shows two labels and does not yet display the notebook's Uncertain outcome

Because of these limits, this model should not be used as the only evidence for any decision about a person's work.

15. Conclusion and Future Work

15.1 Conclusion

This project applied the complete Machine Learning workflow to a real, large text dataset. Against a baseline of 62.76%, three linear models on TF-IDF features reached between 97.92% and 99.95% test accuracy, with tuning by cross-validation. Further testing then showed that a high score on same-source data did not guarantee good behaviour on other human writing.

Error analysis identified four causes, and the final model addressed each with style features, short-text training, wider human data and a validation-chosen threshold. False flags on short human essay snippets fell from 8.5% to 0.4%, and 96.8% of AI essays were still detected. The final model's benchmarks show it is light enough for interactive use, and it runs in a working web application. The most valuable result of the project is therefore not its top score but the process that revealed why that score was not enough.

15.2 Future work

- Add AI-written blogs, news and other non-essay text, so that style is not confused with source.
- Evaluate on a fresh test set that played no part in any decision, ideally with newer AI models.
- Train on more of the available data and widen the hyperparameter search.

- Check and improve probability calibration, for example with reliability plots.
- Compare against a transformer model such as DistilBERT, weighing accuracy against the cost of running it.
- Remove or mask anonymisation tokens such as `STUDENT_NAME` to reduce dataset-specific signals.
- Display the Uncertain outcome in the web application.

References

1. Liang, W., Yuksekgonul, M., Mao, Y., Wu, E., and Zou, J. (2023). GPT detectors are biased against non-native English writers. *Patterns*, 4(7).
2. shanegerami. AI vs Human Text. Kaggle. https://www.kaggle.com/datasets/shanegerami/ai-vs-human-text
3. rtatman. Blog Authorship Corpus. Kaggle. https://www.kaggle.com/datasets/rtatman/blog-authorship-corpus
4. Schler, J., Koppel, M., Argamon, S., and Pennebaker, J. (2006). Effects of age and gender on blogging. *AAAI Spring Symposium on Computational Approaches to Analyzing Weblogs*.
5. alfathterry. BBC Full Text Document Classification. Kaggle. https://www.kaggle.com/datasets/alfathterry/bbc-full-text-document-classification
6. Salton, G., and Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513–523.
7. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
8. Fan, R.-E., Chang, K.-W., Hsieh, C.-J., Wang, X.-R., and Lin, C.-J. (2008). LIBLINEAR: A library for large linear classification. *Journal of Machine Learning Research*, 9, 1871–1874.
9. Cortes, C., and Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.

Acknowledgement. The web application, presentation and this report were prepared with AI coding assistance (Claude Code). Benchmarks and all figures not marked as notebook output were produced from the project's saved files.

Appendix A: Web Application

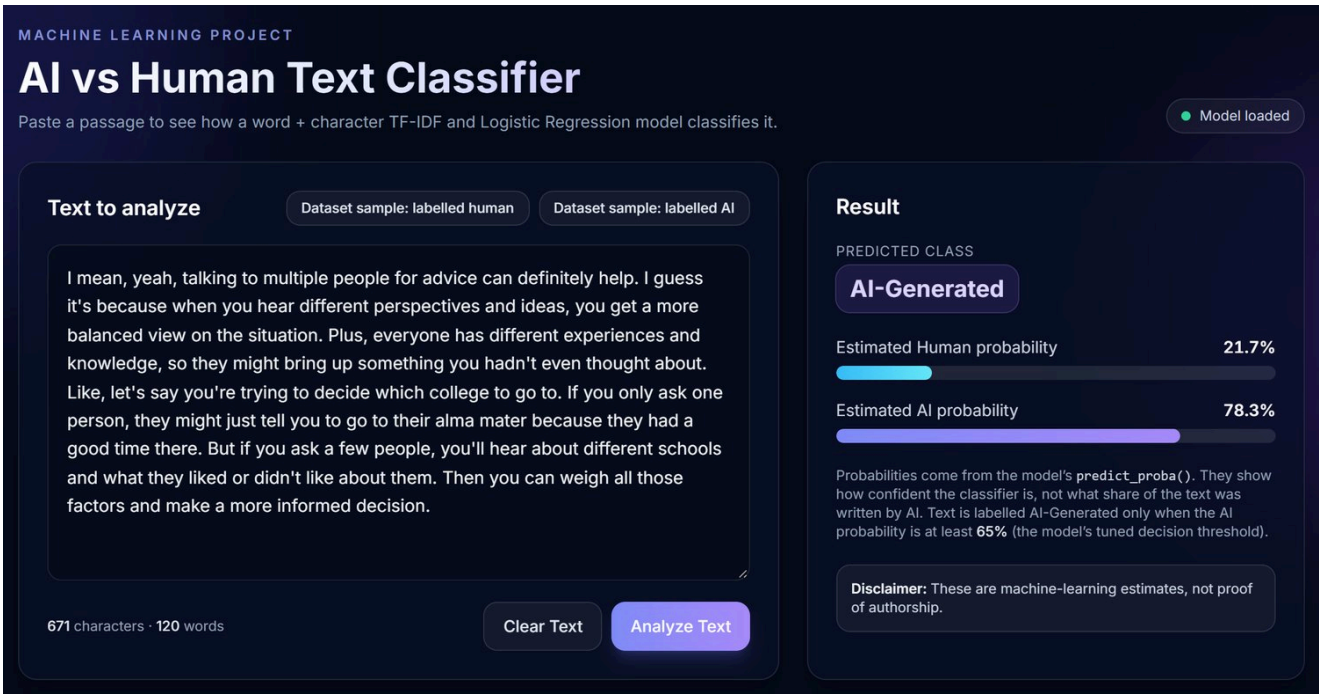


Figure 13. The running application on a dataset sample labelled AI.

Table 29. Application components

Part	Details
Frontend	React, Vite and Tailwind CSS: text input with character and word counts, sample buttons, two probability bars, a disclaimer, session history and an About section
Backend	FastAPI: `POST /predict` and `GET /health`, input validation, CORS for the local frontend
Model loading	joblib, using paths from `backend/.env`. The app registers `clean_text()` before loading, because the pipeline was saved with a reference to it
Prediction	`predict_proba()` from the saved pipeline. No result is hard-coded and nothing is retrained
Missing model	A clear setup message and HTTP 503. Another model is never substituted
Tests	11 automated backend tests, all passing, including a check that API output matches the pipeline's `predict_proba()`

The notebook's introduction mentions a Streamlit deployment. The delivered application uses React and FastAPI instead, which separates the interface from a reusable REST API.

To run on Windows: double-click `run.bat`, which sets up dependencies on first use, starts both servers and opens http://localhost:5173. The application rejects empty text, texts under 50 words and texts over 50,000 characters, each with a readable message.

Appendix B: Reproducibility

Table 30. Project files

File	Purpose
`model_training/notebookc4a5d79b4f.ipynb`	Complete training notebook
`saved models/improved_lr_pipeline.joblib`	Final trained pipeline
`saved models/clean_text.py`	Text cleaning function required by the pipeline
`saved models/improved_model_config.json`	Threshold (0.65) and minimum words (50)
`final_model_comparison.csv`	Phase A comparison metrics
`backend/`, `frontend/`, `run.bat`	Web application
`docs/`	Presentation, pitch notes, this report and the project brief

The notebook uses `random_state=42` throughout. It needs the three Kaggle datasets attached and scikit-learn 1.6.1 to match the saved model. The final model's feature weights in Section 13.2 can be reproduced with `model.named_steps["classifier"].coef_` and the vectorizers' `get_feature_names_out()`.

Appendix C: Coverage of the Course Brief

Table 31. Pipeline stages from the brief and where they are reported

Stage	Section	Stage	Section
1. Problem Definition	2	11. Model Selection	9.1
2. Dataset Acquisition	3	12. Model Training	9.2, 9.3
3. Dataset Understanding	3, 4	13. Hyperparameter Tuning	10
4. Data Quality Check	4.1	14. Model Evaluation	11
5. Exploratory Data Analysis	4.2–4.4	15. Model Comparison	11.1, 11.4, 11.6
6. Data Preprocessing	5	16. Error Analysis	12
7. Feature Engineering	6.1–6.3	17. Interpretation	13
8. Feature Selection	6.4	18. Final Prediction / Demonstration	11.3, Appendix A
9. Train-Test Split	7.2	19. Conclusion	14, 15
10. Baseline Model	8	Benchmarks (added)	11.4, 11.5